

FlowOps

Phase 1 — Five Go Microservices over gRPC

This document is your personal reference for everything built in Phase 1. It covers the idea, the architecture, every service, the proto contracts, and the exact commands used. Read it once and the whole thing comes back.

What FlowOps is

FlowOps is a GitOps deployment engine with full observability. Phase 1 lays the foundation — five Go microservices that talk to each other over gRPC, simulating a realistic order-processing system. Later phases add the GitOps reconciler engine, Kubernetes, and live Grafana dashboards.

The five services and how they connect



Request flow: User → API Gateway → Auth → Orders → Payments → Notifications

Every request enters through the API Gateway (HTTP). The gateway calls Auth to validate the token, then calls Orders. Orders calls Payments to charge the user, then calls Notifications to confirm. Each service does one job only.

1. Why proto files first

Before writing any service code, you define the **contracts** in .proto files. A proto file says: *"this service accepts X input and returns Y output."* The protoc tool then auto-generates the Go structs and gRPC interfaces — you never write this boilerplate manually.

Why not just use JSON over HTTP? You could. But proto gives you: (1) binary format — faster than text, (2) enforced types — wrong type = compile error, (3) auto-generated code across all services from one source of truth.

Proto file anatomy

Every .proto file has the same four parts:

- **syntax** — always "proto3", first line always
- **package** — namespace for this proto
- **option go_package** — where generated Go code lands
- **messages** — request/response structs (like Go structs)
- **service + rpc** — the function you can call remotely

Example — payments.proto

```
syntax = "proto3";
package payments;
option go_package = "github.com/myselfkunal/FlowOps/proto/gen/payments";

message ProcessPaymentRequest {
    float amount = 1;           // field type, name, tag number
}

message ProcessPaymentResponse {
    bool success = 1;
}

service PaymentService {
    rpc ProcessPayment (ProcessPaymentRequest) returns (ProcessPaymentResponse);
}
```

The = 1 after each field is a tag — not a value, just an ID proto uses internally. First field is always 1, second is 2, and so on.

Generating Go code from proto

Run this command once per proto file. It creates two Go files: .pb.go (structs) and _grpc.pb.go (server/client interfaces).

```
# Create output folder first
mkdir proto/gen/payments

# Generate
protoc --proto_path=proto \
```

```
--go_out=proto/gen/payments --go_opt=paths=source_relative \  
--go-grpc_out=proto/gen/payments --go-grpc_opt=paths=source_relative \  
proto/payments.proto
```

Each service gets its own subfolder in proto/gen/ to avoid package conflicts. Do not put all generated files in one flat folder.

2. Go module setup

FlowOps is a **single Go module** at the root. This means one `go.mod` covers all five services and the shared proto code. Run this once at the project root:

```
go mod init github.com/myselfkunal/FlowOps
```

Why github.com in the name? It is just a naming convention — nothing gets created on GitHub. Using your GitHub URL as a prefix guarantees the module name is globally unique. You could name it anything, but this is the Go community convention.

Install dependencies

```
go get google.golang.org/grpc
go get google.golang.org/protobuf
```

3. Project folder structure

Every file in the project has a reason. Here is the full structure after Phase 1:

```
FlowOps/
├── proto/
│   ├── payments.proto
│   ├── auth.proto
│   ├── orders.proto
│   ├── notifications.proto
│   └── gen/
│       ├── payments/
│       ├── auth/
│       ├── orders/
│       └── notifications/
├── services/
│   ├── api-gateway/main.go
│   ├── auth/main.go
│   ├── orders/main.go
│   ├── payments/main.go
│   └── notifications/main.go
├── go.mod
└── go.sum
```

proto/ — all contract files live here, never inside a service folder.

proto/gen/ — auto-generated Go code, one subfolder per service. Never edit these files.

services/ — one folder per service, each with its own main.go.

go.mod / go.sum — single module definition at root.

4. Services reference

Quick reference for all five services:

Service	Port	Protocol	Job	Calls
API Gateway	:8080	HTTP	Front door, routes requests	Auth, Orders
Auth	:50051	gRPC	Validates token	—
Orders	:50052	gRPC	Creates orders	Payments, Notifs
Payments	:50053	gRPC	Processes payment	—
Notifications	:50054	gRPC	Notifies user	—

Build order

Always build services from the **least dependent to most dependent**:

1. Payments — no dependencies
2. Notifications — no dependencies
3. Auth — no dependencies
4. Orders — calls Payments and Notifications
5. API Gateway — calls Auth and Orders

5. gRPC server pattern

Every gRPC service (Auth, Orders, Payments, Notifications) follows the exact same pattern. Memorise this once and it applies to all of them.

Step 1 — Create a struct that embeds the unimplemented server

```
type PaymentServer struct {
    pb.UnimplementedPaymentServiceServer
}
```

UnimplementedPaymentServiceServer is a safety net from the generated code. If you forget to implement a function, it returns an error instead of crashing.

Step 2 — Implement the interface function

```
func (s *PaymentServer) ProcessPayment(
    ctx context.Context,
    req *pb.ProcessPaymentRequest,
) (*pb.ProcessPaymentResponse, error) {

    log.Printf("Processing payment: %v", req.Amount)
    return &pb.ProcessPaymentResponse{Success: true}, nil
}
```

Step 3 — Start the gRPC server in main()

```
func main() {
    lis, err := net.Listen("tcp", ":50053")
    if err != nil {
        log.Fatalf("Failed to listen: %v", err)
    }

    grpcServer := grpc.NewServer()
    pb.RegisterPaymentServiceServer(grpcServer, &PaymentServer{})

    log.Println("Payments service running on :50053")
    grpcServer.Serve(lis)
}
```

6. Orders service — acting as a gRPC client

Orders is different from the others. It is both a **server** (receives CreateOrder calls) and a **client** (calls Payments and Notifications). The clients are stored inside the server struct so connections are created once and reused.

Server struct with embedded clients

```
type OrderServer struct {
    pb.UnimplementedOrderServiceServer
    paymentsClient    paymentspb.PaymentServiceClient
    notificationsClient notifpb.NotificationsServiceClient
}
```

Setting up connections in main()

```
paymentsConn, _ := grpc.NewClient("localhost:50053",
    grpc.WithTransportCredentials(insecure.NewCredentials()),
)
notifConn, _ := grpc.NewClient("localhost:50054",
    grpc.WithTransportCredentials(insecure.NewCredentials()),
)

orderServer := &OrderServer{
    paymentsClient:    paymentspb.NewPaymentServiceClient(paymentsConn),
    notificationsClient: notifpb.NewNotificationsServiceClient(notifConn),
}
```

CreateOrder flow

```
func (s *OrderServer) CreateOrder(ctx context.Context, req *pb.OrderRequest) (*pb.OrderResponse, error) {
    orderID := fmt.Sprintf("order-%d", time.Now().Unix())

    // call payments
    payRes, err := s.paymentsClient.ProcessPayment(ctx, &paymentspb.ProcessPaymentRequest{
        Amount: req.Amount,
    })
    if err != nil || !payRes.Success {
        return &pb.OrderResponse{Success: false}, nil
    }

    // call notifications
    s.notificationsClient.SendNotifications(ctx, &notifpb.NotificationsRequest{
        UserId: req.UserId,
        Success: true,
    })

    return &pb.OrderResponse{OrderId: orderID, Success: true}, nil
}
```


7. API Gateway — HTTP to gRPC

The gateway is the only service without a proto file. It is a plain HTTP server that translates incoming JSON requests into gRPC calls. No `grpc.NewServer()` here — only clients.

Key pattern — global clients

```
var (
    authClient  authpb.AuthServiceClient
    ordersClient orderspb.OrderServiceClient
)
```

Handler function

```
func handleOrder(w http.ResponseWriter, r *http.Request) {
    var req OrderRequest
    json.NewDecoder(r.Body).Decode(&req)

    // validate token
    authRes, err := authClient.Validate(r.Context(), &authpb.AuthRequest{
        AuthToken: req.Token,
    })
    if err != nil || !authRes.Success {
        w.WriteHeader(http.StatusUnauthorized)
        json.NewEncoder(w).Encode(map[string]string{"error": "unauthorized"})
        return
    }

    // create order
    orderRes, err := ordersClient.CreateOrder(r.Context(), &orderspb.OrderRequest{
        UserId:    req.UserID,
        ItemName:  req.ItemName,
        Amount:    req.Amount,
    })
    if err != nil {
        w.WriteHeader(http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]interface{}{
        "success": true,
        "order_id": orderRes.OrderId,
    })
}
```

Critical: always use `r.Context()` not `context.Background()` when making gRPC calls from the handler. `r.Context()` carries trace context forward. `context.Background()` breaks the trace chain.

8. Testing the full chain

Start all five services in separate terminals, then send a request:

```
# Start each in its own terminal
go run services/payments/main.go
go run services/notifications/main.go
go run services/auth/main.go
go run services/orders/main.go
go run services/api-gateway/main.go
```

Test with curl (PowerShell):

```
curl.exe -X POST http://localhost:8080/order -H "Content-Type: application/json" -d '{"token":"valid-token-123"}'

# Expected response
{"order_id":"order-1780816813","success":true}
```

What this proves:

- API Gateway received the HTTP request
- Auth validated the token "valid-token-123"
- Orders created the order with a timestamp-based ID
- Payments processed the amount
- Notifications sent the confirmation

Troubleshooting

Symptom	Cause	Fix
curl: Failed to connect	Gateway not running	Check all 5 terminals
{error: unauthorized}	Wrong token / JSON escaping	Use backtick escaping in PowerShell
undefined: pb.Something	Wrong import path	Check proto gen subfolder path
protoc not found in VS Code	PATH not refreshed	Add PATH to VS Code settings JSON

9. Key lessons from Phase 1

Proto files define contracts before code.

Write the proto, generate the Go code, then implement. Never implement first and figure out the contract later.

One Go module at the root.

Don't create separate `go.mod` files per service. One module at root means all services can import each other and the shared proto code cleanly.

Generated code goes in subfolders.

Put each service's generated proto code in `proto/gen/servicename/`. Mixing them in one flat `proto/gen/` folder causes package conflicts.

Build from least dependent to most.

Payments and Notifications first, gateway last. This way you can test each service in isolation before wiring them together.

`context.Background()` breaks traces.

Always pass `r.Context()` or the incoming `ctx` when making downstream calls. Creating a new `context.Background()` breaks the trace chain — you'll lose the connection between spans in Jaeger.

Blank screen paralysis is normal.

Never ask yourself 'how do I build this service.' Always ask 'what is the single next physical action I can take.' Make the folder. Run `go mod init`. Write `package main`. The code follows one line at a time.

Phase 1 complete. Five services, gRPC contracts, full call chain verified end to end.

Next: Phase 2 — OpenTelemetry observability, Jaeger traces, Prometheus metrics, Grafana dashboards.